

CYBERSECURITY PORTFOLIO

Home-Lab Series | Linux Firewall & Packet Filtering

iptables

Linux Firewall & Router

NAT · Forwarding · Segmentation · Logging · SSH Hardening

Ihar Shcharbitski · 21.05.2026

UBUNTU 26.04

FIREWALL / ROUTER

iptables · NAT · routing

PARROT 7.1 KDE

CLIENT / ATTACKER

nmap · ping · ssh · curl

Oracle VirtualBox | Ubuntu 26.04 | Parrot 7.1 KDE Security

01 LAB OVERVIEW

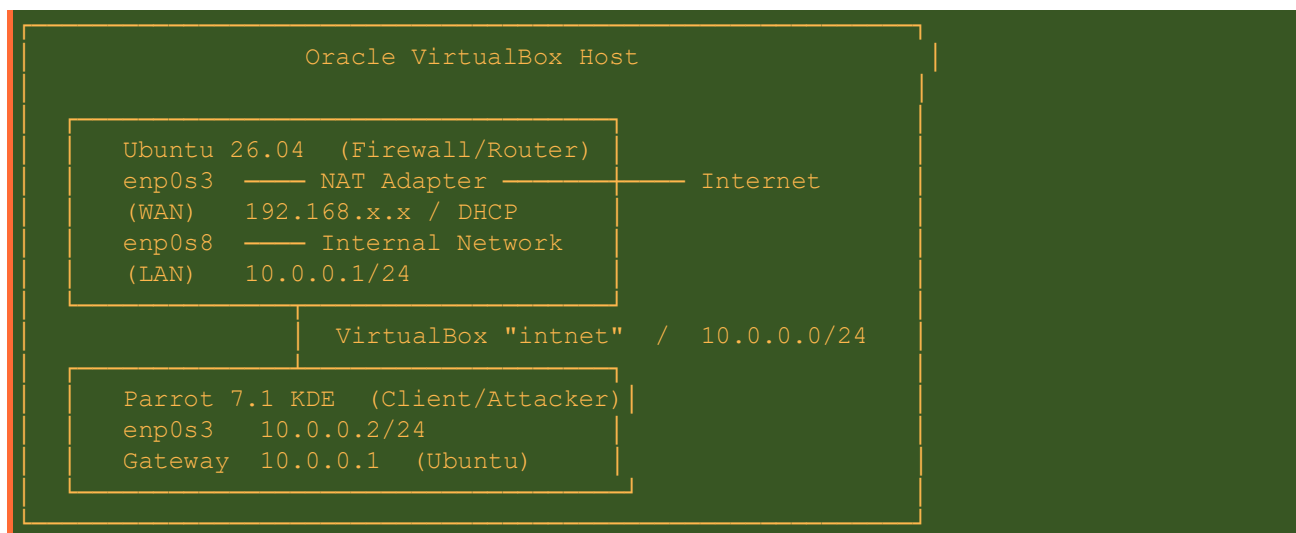
Goal of the Lab

In this lab I want to demonstrate practical knowledge in the following areas:

- **Packet flow:** Understanding how packets traverse a Linux router.
- **Stateful filtering:** Allowing reply traffic without opening unnecessary inbound ports.
- **NAT / Masquerade:** How home routers hide private IPs — built from scratch.
- **Logging:** Capturing every dropped or suspicious packet in real time.
- **SSH hardening:** Locking down remote access using keys, rate limiting, and Fail2ban.

Architecture

Ubuntu acts as a gateway between Parrot and the internet. All Parrot traffic passes through Ubuntu's iptables rules — giving full visibility and control over every packet.



Notes

Throughout this document I list my full knowledge on each topic. However, in the practical implementation I do not use every technique covered in the theory — this is intentional. The concepts section gives the complete picture; the steps section shows what I actually built.

Security and logging are treated with particular care. After scripting, I validate each rule empirically rather than relying on assumptions.

Since I do not yet have advanced red-team skills, the validation section uses the tools available to me — primarily `nmap`, `ping`, `curl`, and SSH probes from Parrot. The firewall logs on Ubuntu provide independent confirmation that every rule fires as intended.

02 NECESSARY INFORMATION — IPTABLES CORE CONCEPTS

Netfilter is a framework built directly into the Linux kernel. It provides hooks at specific points in the packet processing path. iptables is the userspace tool that writes rules into those Netfilter hooks.

iptables consists of five components:

- **Tables** — organize and categorize firewall rules. The four tables are: **filter** (default — allow/drop), **nat** (address translation), **mangle** (packet header modification), **raw** (bypass connection tracking). In this lab I primarily use **filter** and **nat**.
- **Chains** — group rules applied to a specific traffic type. There are two categories: **built-in chains** (INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING) and **user-defined chains** (custom, for organization).
- **Rules** — define the criteria for filtering network traffic and the action to take for matching packets.
- **Matches** — specify the criteria that determine whether a rule applies to a particular packet (e.g., protocol, source IP, destination port, connection state).
- **Targets** — specify the action for packets that match (ACCEPT, DROP, REJECT, LOG, MASQUERADE, DNAT, SNAT).

Target	Behavior
ACCEPT	Allow the packet through.
DROP	Silently discard. The sender receives no response — preferred for stealth.
REJECT	Discard and send an ICMP error back to the sender.
LOG	Write packet details to syslog. Continues to the next rule — does NOT stop processing.
MASQUERADE	Replace source IP with the firewall's outbound IP. Used for NAT on dynamic IPs.
DNAT	Rewrite the destination IP/port. Used to redirect traffic.
SNAT	Rewrite the source IP/port. Used when the WAN IP is static.

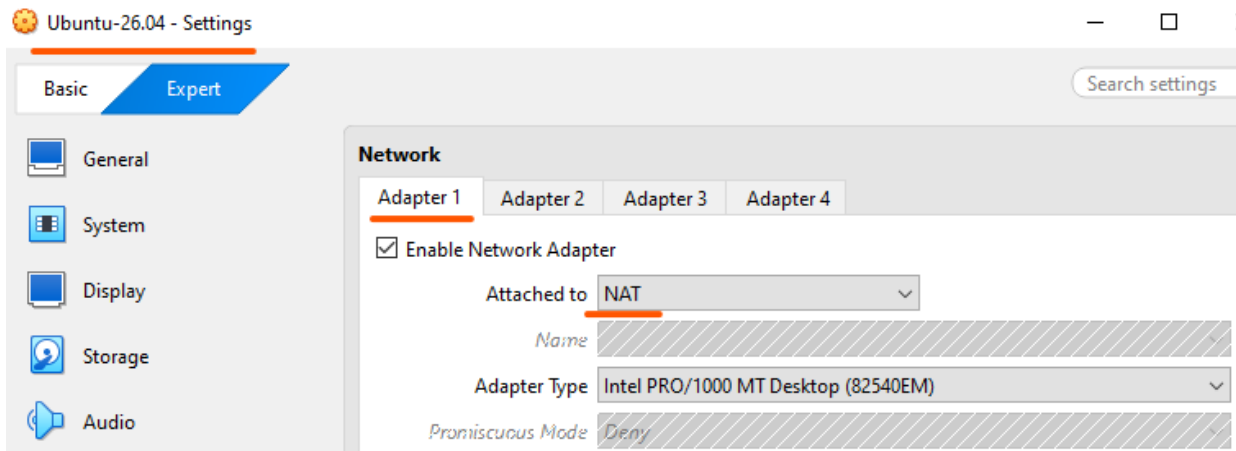
Stateful Filtering: iptables tracks the state of every connection: NEW (first packet), ESTABLISHED (reply traffic for an accepted connection), RELATED (related to an existing connection), INVALID (no known connection — drop these). The rule '-m state --state ESTABLISHED,RELATED -j ACCEPT' allows all reply traffic without opening unnecessary inbound ports.

03 PREPARE ENVIRONMENT

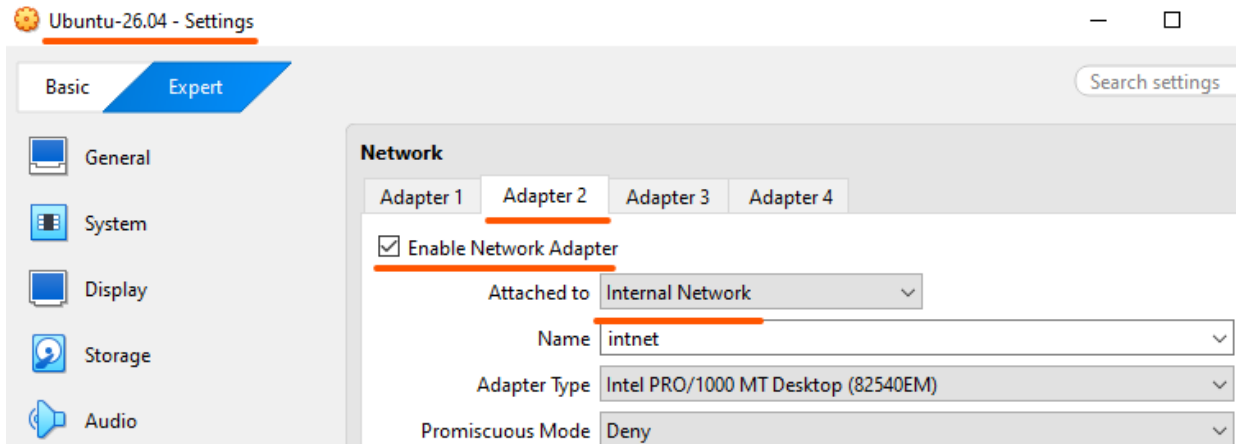
VirtualBox Network Setup

Ubuntu requires two network adapters — one for internet access (WAN) and one for the private segment shared with Parrot (LAN). Parrot requires only one adapter pointing to the same internal network.

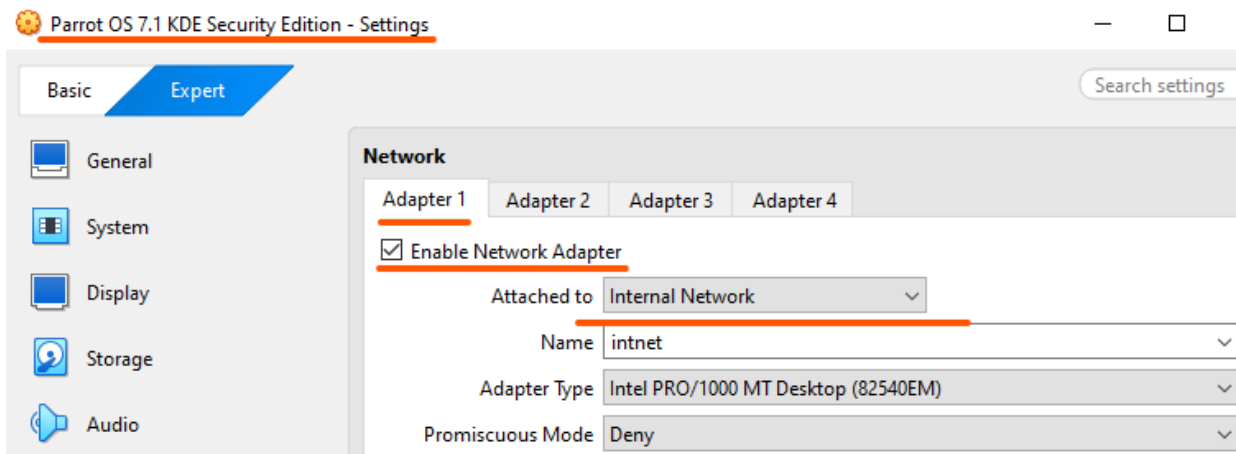
Step 3.1 — Ubuntu's 1st Adapter — WAN (NAT) [\[Oracle VirtualBox\]](#)



Step 3.2 — Ubuntu's 2nd Adapter — LAN (Internal Network: intnet) [\[Oracle VirtualBox\]](#)



Step 3.3 — Parrot's Only Adapter — LAN (Internal Network: intnet) [\[Oracle VirtualBox\]](#)



The Internal Network name must be identical on both VMs — 'intnet' is case-sensitive in VirtualBox.

Identify Network Interfaces

Step 3.4 — Identify interfaces on Ubuntu

What it does: Lists all network interfaces and their current state. Confirms two interfaces are present beyond loopback: the WAN adapter (enp0s3) connected to VirtualBox NAT, and the LAN adapter (enp0s8) on the internal network.

```
shcherba@Ubuntu-26-04:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:63:a1:97 brd ff:ff:ff:ff:ff:ff
   altname enx08002763a197
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:16:fc:a6 brd ff:ff:ff:ff:ff:ff
   altname enx08002716fca6
```

Step 3.5 — Identify interfaces on Parrot

What it does: Same command on Parrot — confirms the single internal adapter is present and ready for static IP assignment.

```
~ : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
[user@parrot]~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 08:00:27:76:7f:09 brd ff:ff:ff:ff:ff:ff
   altname enx080027767f09
[user@parrot]~$
```

Assign Static IP to Ubuntu's LAN Interface

Step 3.6 — Configure /etc/netplan — Static IP on enp0s8

The WAN interface (enp0s3) uses DHCP — VirtualBox's NAT assigns its IP automatically. The LAN interface (enp0s8) needs a fixed IP because Parrot will use it as its default gateway.

```
shcherba@Ubuntu-26-04: ~ — sudo vim /etc/netplan/01-network-manager-all.yaml

network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses:
        - 10.0.0.1/24
```

Step 3.7 — Apply the Netplan Configuration

What it does: Reads the YAML configuration and applies it to the running kernel. After this, enp0s8 will hold the static IP 10.0.0.1/24.

```
shcherba@Ubuntu-26-04:~$ sudo netplan apply
systemd-networkd is not running, output might be incomplete.
Failed to reload network settings: Unit dbus-org.freedesktop.network1.service not found.
Falling back to a hard restart of systemd-networkd.service
```

Troubleshooting — systemd-networkd not running:

The error above indicates that the 'networkd' renderer is not active. The fix is straightforward:

Step 3.8 — Start systemd-networkd

What it does: Starts and enables the networkd service that netplan depends on. After this, 'sudo netplan apply' completes without errors.

```
shcherba@Ubuntu-26-04:~$ sudo systemctl start systemd-networkd
shcherba@Ubuntu-26-04:~$ sudo netplan apply
shcherba@Ubuntu-26-04:~$
```

Assign Static IP to Parrot's LAN Interface

Step 3.9 — Configure /etc/network/interfaces — Static IP on Parrot

Parrot uses the older ifupdown networking system. The most important line here is 'gateway 10.0.0.1' — this points all Parrot's outbound traffic to Ubuntu.

```
File Edit View Bookmarks Plugins Settings H

1 source /etc/network/interfaces.d/*
2 auto enp0s3
3 iface enp0s3 inet static
4   address 10.0.0.2
5   netmask 255.255.255.0
6   gateway 10.0.0.1
7   dns-nameservers 8.8.8.8 1.1.1.1
8
```

Step 3.10 — Restart Networking and Verify Connectivity

What it does: Applies the static IP configuration. The ping confirms both machines can communicate over the internal network — a prerequisite before any firewall rules are applied.

```
[user@parrot]~  
$ sudo systemctl restart networking  
[user@parrot]~  
$ ping -c 4 10.0.0.1  
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.  
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.179 ms  
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.211 ms  
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.208 ms  
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.191 ms  
  
--- 10.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3031ms  
rtt min/avg/max/mdev = 0.179/0.197/0.211/0.013 ms
```

If ping shows 100% packet loss, should verify that both VMs are using the same Internal Network name in VirtualBox. A single character difference ('intnet' vs 'Intnet') will break communication.

04 BUILDING THE FIREWALL

The entire firewall is built inside a single shell script — `/etc/firewall.sh`. Working from a script means rules can be flushed and cleanly reapplied in seconds, which is essential during development and troubleshooting.

Step 4.1 — Define WAN, LAN, and LAN_SUBNET Variables

What it does: Variables make the script portable — changing the interface name in one place updates every rule that references it.

```
#!/bin/bash

WAN="enp0s3"
LAN="enp0s8"
LAN_SUBNET="10.0.0.0/24"
```

Step 4.2 — Flush Previous Rules, User Chains, Counters, NAT Table Rules, and NAT User Chains

What it does: Clears any previously loaded rules from all relevant tables before applying fresh ones. This prevents conflicts and ensures a known starting state every time the script runs.

```
echo "Flushing existing rules..."
iptables -F #flush filter rules
iptables -X #flush user-defined chains in filter-table
iptables -Z #flush counters
iptables -t nat -F #flush nat table rules
iptables -t nat -X #flush user-defined chains in nat table
```

Step 4.3 — Default Policies — Deny by Default

What it does: Sets the fallback behavior for each chain when no rule matches. INPUT and FORWARD default to DROP — anything not explicitly permitted is silently discarded. OUTPUT is ACCEPT because Ubuntu's own outbound traffic needs to flow freely.

```
#Default Policies
iptables -P INPUT -j DROP #DROP packets, which go to router
iptables -P OUTPUT -j DROP #DROP packets, which go out from router
iptables -P FORWARD -j DROP #DROP packets, which router
```

Deny by Default: Setting the default policy to DROP is a fundamental security principle: explicitly allow only what is needed, and block everything else. This is the opposite of the system's default ACCEPT stance.

Step 4.4 — Allow Loopback Traffic

What it does: The loopback interface (lo) carries internal OS traffic between processes on the same machine. Breaking loopback can crash system services, so it must always be permitted regardless of policy.


```
#Allow loopback traffic
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT
```

Step 4.5 — Allow ESTABLISHED and RELATED Traffic

What it does: Enables stateful filtering — any packet that belongs to a connection already permitted inbound is automatically accepted. Without this rule, the DROP policy would block all reply traffic, including responses to Ubuntu's own outbound requests.

```
iptables -A INPUT -m --state ESTABLISHED,RELATED -j ACCEPT
```

Step 4.6 — Log INVALID Packets and DROP Them

What it does: INVALID packets do not belong to any tracked connection — they are often crafted probes, scan artifacts, or malformed traffic. The LOG rule fires first (it does not stop processing), writing details to syslog with the prefix FW_INVALID:. The DROP rule then silently discards them.

```
iptables -A INPUT -m state --state INVALID -j LOG --log-prefix "FW_INVALID: " --log-level 4
iptables -A INPUT -m state --state INVALID -j DROP
```

Step 4.7 — Allow ICMP Echo Requests from LAN Only

What it does: Allows ping from the LAN segment (Parrot) to Ubuntu. This is essential for connectivity testing. The '-i \$LAN' restriction ensures no ping requests arrive from the WAN interface.

```
iptables -A INPUT -i $LAN -p ICMP --icmp-type echo-request -j ACCEPT
```

Step 4.8 — Log SSH Attempts

What it does: Logs every new SSH connection attempt arriving from the LAN. Because LOG does not stop processing, the packet continues to the next rule. This produces an audit trail of all connection attempts without affecting the allow/deny decision.

```
iptables -A INPUT -i $LAN -p tcp --dport 22 -m state --state NEW -j LOG --log-prefix "FW_SSH_ATTEMPT: " --log-level 4
```

Step 4.9 — Allow Rate-Limited SSH]

What it does: Permits new SSH connections from the LAN at a maximum rate of 3 per minute, with an initial burst of 5. The '--limit-burst 5' allows the first 5 connections immediately; thereafter the rate is throttled. This slows brute-force attempts significantly without blocking legitimate use.

```
iptables -A INPUT -i $LAN -p tcp --dport 22 -m state --state NEW -m limit --limit 3/min --limit-burst 5 -j ACCEPT
```

Step 4.10 — Log and Drop SSH Connections Exceeding the Rate Limit

What it does: Any SSH connection that did not match the rate-limit ACCEPT rule above falls through to these two rules. The packet is first logged with the FW_SSH_RATELIMIT: prefix, then dropped. Rules are evaluated top-to-bottom, so this only fires after the rate limit is exceeded.

```
iptables -A INPUT -i $LAN -p tcp --dport 22 -m state --state NEW -j LOG --log-prefix "FW_SSH_RATELIMIT: " --log-level 4
iptables -A INPUT -i $LAN -p tcp --dport 22 -m state --state NEW -j DROP
```

Rules are read top to bottom. The chain checks the rate limit first; those within the limit are accepted. Those exceeding it fall through to this LOG + DROP pair.

Step 4.11 — Log and Block WAN SSH Completely

What it does: Prevents any SSH connection attempt arriving on the WAN interface from reaching the daemon. The LOG prefix FW_SSH_WAN_BLOCK: records the source IP for monitoring — useful for detecting external scan activity.

```
iptables -A INPUT -i $WAN -p tcp --dport 22 -j LOG --log-prefix "FW_SSH_WAN_BLOCK: " --log-level 4
iptables -A INPUT -i $WAN -p tcp --dport 22 -j DROP
```

Step 4.12 — Allow Return Traffic to LAN (FORWARD chain)

What it does: Stateful rule for forwarded traffic — allows reply packets from the internet back to Parrot for connections that Parrot already initiated. Without this, Parrot's requests would leave but replies would be dropped.

```
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
```

Step 4.13 — Allow LAN to Reach the Internet (FORWARD chain)

What it does: Allows Parrot to initiate new connections to the internet through Ubuntu's WAN interface. This is the outbound forwarding rule that, combined with NAT masquerade, gives Parrot internet access.

```
iptables -A FORWARD -i $LAN -o $WAN -m state --state NEW -j ACCEPT
```

Step 4.14 — Log All FORWARD DROPS

What it does: Appended at the bottom of the FORWARD chain — this rule only fires if a packet was not matched by any of the ACCEPT rules above. It logs the dropped packet before the chain's default DROP policy discards it. Essential for diagnosing blocked traffic.

```
iptables -A FORWARD -j LOG --log-prefix "FW_FORWARD_DROP: " --log-level 4
```

Step 4.15 — Block HTTP (Port 80) While Allowing HTTPS (Port 443)

What it does: '-I FORWARD 1' inserts this rule at position 1 — the very top of the chain — ensuring it is evaluated before the general ACCEPT rule at a lower position. This forces all web traffic through HTTPS by blocking plaintext HTTP at the firewall level.

```
iptables -I FORWARD -i $LAN -o $WAN -p tcp --dport 80 -j LOG --log-prefix "FW_HTTP_BLOCKED: "
iptables -I FORWARD -i $LAN -o $WAN -p tcp --dport 80 -j DROP
```

Used -I (insert) not -A (append). With -A, the rule goes to the bottom — after the general ACCEPT rule. Since the first matching rule wins, the ACCEPT would fire first and the block would never trigger.

Step 4.16 — Apply NAT — IP Masquerade

What it does: Works in the nat table's POSTROUTING chain — the last processing hook before a packet leaves the machine. '-s \$LAN_SUBNET' matches packets originating from Parrot's private

range. MASQUERADE replaces the source IP with Ubuntu's dynamic WAN IP, making the traffic routable on the internet.

```
iptables -t nat -A POSTROUTING -s $LAN_SUBNET -o $WAN -j MASQUERADE
```

NAT Masquerade: Private IPs (10.0.0.x) are not routable on the internet. MASQUERADE rewrites the source IP to Ubuntu's public WAN IP on every outbound packet and reverses the translation for inbound replies. The kernel maintains a connection tracking table for this. This is exactly how a home router works.

Step 4.17 — Log All Dropped Packets from the Outside (WAN)

What it does: Catches any unsolicited inbound packet arriving on the WAN interface that was not matched by a previous ACCEPT rule. Logging these is particularly valuable from a security monitoring perspective — it reveals external scan activity and connection attempts.

```
iptables -A INPUT -j LOG --log-prefix "FW_INPUT_DROP: " --log-level 4
```

Step 4.18 — Time-Based Rule — Restrict Internet Access on Specific Days

What it does: Demonstrates the 'time' match module, which allows rules to be active only during specific time windows. Here, all LAN-to-WAN forwarding is blocked on weekends. Not used in active production for this lab — included to demonstrate the capability.

```
iptables -A INPUT -i $LAN -o $WAN -m time --weekdays Sat,Sun -j DROP
```

Step 4.19 — Final Lines — Echo Completion Message and List Rules

What it does: The echo confirms the script completed. 'iptables -L -v --line-numbers' lists every active rule with packet/byte counters and line numbers — useful for immediate verification after applying the script.

```
echo "Firewall rules applied"
iptables -L -v --line-numbers
```

Apply the Firewall Script

Step 4.20 — Give Execute Permission and Run the Script

What it does: chmod +x grants execute permission. Running the script as root applies all rules to the live kernel. The output confirms each section was processed.

```
shcherba@Ubuntu-26-04:~$ sudo chmod +x /etc/firewall.sh
```

```
shcherba@Ubuntu-26-04:~$ sudo /etc/firewall.sh
Flushing existing rules...
```

```

Firewall rules applied
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1      0      0 ACCEPT    all  --  lo      any      anywhere          anywhere
2      0      0 LOG       all  --  any     any      anywhere          anywhere          state INVALID LOG le
vel warn prefix "FW_INVALID: "
3      0      0 DROP      all  --  any     any      anywhere          anywhere          state INVALID
4      0      0 ACCEPT    icmp --  enp0s8  any      anywhere          anywhere          icmp echo-request
5      0      0 LOG       tcp  --  enp0s8  any      anywhere          anywhere          tcp dpt:ssh state NE
W LOG level warn prefix "FW_SSH_ATTEMPT: "
6      0      0 ACCEPT    tcp  --  enp0s8  any      anywhere          anywhere          tcp dpt:ssh state NE
W limit: avg 3/min burst 5

```

Enable IP Forwarding

The Linux kernel is not a router by default. IP forwarding must be explicitly enabled — without it, packets arriving on enp0s8 from Parrot will never be forwarded to enp0s3.

Step 4.21 — Enable IP Forwarding for the Current Session

What it does: Writes '1' directly into the kernel's virtual filesystem at /proc. This activates IP forwarding immediately without a reboot. The /proc filesystem is the kernel's live interface — changes here take effect instantly.

```

shcherba@Ubuntu-26-04:~$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1
shcherba@Ubuntu-26-04:~$

```

Step 4.22 — Enable IP Forwarding Permanently via /etc/sysctl.conf

What it does: /etc/sysctl.conf stores kernel parameter overrides that are loaded at boot. The key-value entry 'net.ipv4.ip_forward=1' makes forwarding survive reboots.

```

shcherba@Ubuntu-26-04: ~ — sudo vim /etc/sysctl.conf
net.ipv4.ip_forward=1

```

Step 4.23 — Apply Changes and Verify

What it does: 'sysctl -p' reads /etc/sysctl.conf and sends each parameter directly into the running kernel — no reboot required. The cat command confirms the value is now 1.

```

shcherba@Ubuntu-26-04:~$ sudo sysctl -p
net.ipv4.ip_forward = 1

shcherba@Ubuntu-26-04:~$ cat /proc/sys/net/ipv4/ip_forward
1

```

Persist Rules Across Reboots

iptables rules live in kernel RAM and are lost on every reboot. The following steps save them to disk and configure automatic reloading.

Step 4.24 — Install iptables-persistent

What it does: Installs the iptables-persistent package, which provides a systemd service that automatically loads saved rules at boot. During installation it prompts to save the current ruleset — select Yes.

```
shcherba@Ubuntu-26-04:~$ sudo apt install iptables-persistent -y
Installing:
  iptables-persistent

Installing dependencies:
  netfilter-persistent
```

Step 4.25 — Save Rules Manually

What it does: Exports the current iptables ruleset in a machine-readable format and writes it to /etc/iptables/rules.v4. Run this command every time the ruleset is modified.

```
shcherba@Ubuntu-26-04:~$ sudo iptables-save | sudo tee /etc/iptables/rules.v4
# Generated by iptables-save v1.8.11 (nf_tables) on Wed May 20 20:16:15 2026
...
# Completed on Wed May 20 20:16:15 2026
```

Step 4.26 — Enable Auto-Load on Reboot

What it does: The saved rules file is useless without a mechanism to reload it. This command tells systemd to start the netfilter-persistent service at every boot, which reads /etc/iptables/rules.v4 and restores the ruleset.

```
shcherba@Ubuntu-26-04:~$ sudo systemctl enable netfilter-persistent
Synchronizing state of netfilter-persistent.service with SysV service script with /usr/lib/systemd/systemd-sysv-in
stall.
Executing: /usr/lib/systemd/systemd-sysv-install enable netfilter-persistent
shcherba@Ubuntu-26-04:~$
```

Routing Table and Connectivity Verification

Step 4.27 — View Ubuntu's Routing Table

What it does: Displays Ubuntu's current routing table — the map the kernel uses to decide where to send each packet.

```
shcherba@Ubuntu-26-04:~$ ip route show
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.0.0/24 dev enp0s8 proto kernel scope link src 10.0.0.1
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
192.168.100.1 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
```

Output breakdown:

- **Default route:** 'default via 10.0.2.2 dev enp0s3' — all traffic without a more specific match exits through the VirtualBox NAT gateway.
- **LAN network:** '10.0.0.0/24 dev enp0s8' — Ubuntu's LAN subnet is directly reachable on enp0s8. src 10.0.0.1 is Ubuntu's LAN IP.
- **VirtualBox NAT network:** '10.0.2.0/24 dev enp0s3' — Ubuntu's WAN subnet. 10.0.2.15 is Ubuntu's WAN IP.
- **Gateway route:** '10.0.2.2' is the VirtualBox NAT router — all internet-bound traffic passes through it.

- **Special route:** Traffic to 192.168.100.1 routes via the NAT gateway (not directly reachable from the VM).

Overall picture: Parrot (10.0.0.x) → Ubuntu LAN (10.0.0.1) → Ubuntu WAN (10.0.2.15) → VirtualBox NAT (10.0.2.2) → Internet

Step 4.28 — End-to-End Connectivity Tests

Parrot → Ubuntu (LAN)

What it does: Verifies that Parrot can reach Ubuntu's LAN interface over the internal network.

```
[user@parrot]~  
$ ping -c 4 10.0.0.1  
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.  
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.182 ms  
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.202 ms  
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.202 ms  
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.193 ms
```

Parrot → Internet (through Ubuntu's NAT)

What it does: Sends ICMP packets from Parrot to Google's DNS server. These traverse the full path: Parrot → Ubuntu → NAT masquerade → VirtualBox → Internet. Successful replies confirm that both forwarding and NAT are working.

```
[user@parrot]~  
$ ping -c 4 8.8.8.8  
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=62 time=17.2 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=62 time=18.7 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=62 time=17.1 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=62 time=11.2 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3037ms  
rtt min/avg/max/mdev = 11.211/16.020/18.666/2.849 ms
```

DNS Resolution Check

What it does: 'nslookup' queries a DNS server to resolve 'google.com' to its IP address. A successful response confirms that DNS traffic is being forwarded correctly through Ubuntu's firewall.

```
[user@parrot]~  
$ nslookup google.com  
Server: 192.168.100.1  
Address: 192.168.100.1#53  
  
Non-authoritative answer:  
Name: google.com  
Address: 142.250.120.102  
Name: google.com  
Address: 142.250.120.138  
Name: google.com  
Address: 142.250.120.100  
Name: google.com  
Address: 142.250.120.139  
Name: google.com  
Address: 142.250.120.101  
Name: google.com
```

Trace Full Path: Parrot → Internet

What it does: Maps every hop between Parrot and the destination. The first hop must be 10.0.0.1 (Ubuntu's LAN IP), the second 10.0.2.2 (VirtualBox NAT gateway), followed by internet hops. This confirms: LAN routing, NAT, forwarding, and firewall pass-through are all functioning correctly.

```
[user@parrot]~  
$ tracert 8.8.8.8  
tracert to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets  
 1  10.0.0.1 (10.0.0.1)  0.191 ms  0.164 ms  0.155 ms  
 2  10.0.2.2 (10.0.2.2)  0.316 ms  0.307 ms  0.335 ms  
 3  10.0.2.2 (10.0.2.2)  8.833 ms  8.890 ms  8.872 ms
```


05 LOGS

Where Logs Go

File	Contents
/var/log/kern.log	Kernel messages — includes all iptables LOG output
/var/log/syslog	Combined system log
/var/log/iptables.log	Dedicated file (after rsyslog configuration)

Step 5.1 — View Firewall Logs in Real Time

What it does: 'tail -f' follows the log file and prints new entries as they arrive. Piping to grep filters for only firewall-related entries using the FW_ prefix. This is a real-time view of every packet decision the firewall makes.

```
shcherbag@Ubuntu-26-04:~$ sudo tail /var/log/kern.log | column -t
2026-05-20T22:23:23.458525+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s8 OUT= MAC=ff:ff:ff:ff:ff:ff:08:00:27:76:7f:09:08:00 SRC=10.0.0.2
DST=255.255.255.255 LEN=2135 TOS=0x00 PREC=0x00 TTL=64 ID=61544 PROTO=UDP SPT=1716 DPT=1716 LEN=2115
2026-05-20T22:23:23.459262+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s8 OUT= MAC=01:00:5e:00:00:fb:08:00:27:76:7f:09:08:00 SRC=10.0.0.2
DST=224.0.0.251 LEN=264 TOS=0x00 PREC=0x00 TTL=1 ID=64138 DF PROTO=UDP SPT=5353 DPT=5353 LEN=244
2026-05-20T22:23:23.459284+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s8 OUT= MAC=01:00:5e:00:00:fb:08:00:27:76:7f:09:08:00 SRC=10.0.0.2
DST=224.0.0.251 LEN=68 TOS=0x00 PREC=0x00 TTL=64 ID=64139 DF PROTO=UDP SPT=49234 DPT=5353 LEN=48
2026-05-20T22:23:26.153181+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.
122 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=26513 PROTO=UDP SPT=123 DPT=46070 LEN=196
2026-05-20T22:23:26.379930+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=91.189.91.11
3 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=30361 PROTO=UDP SPT=123 DPT=35492 LEN=196
2026-05-20T22:23:26.725056+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=91.189.91.11
2 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=22638 PROTO=UDP SPT=123 DPT=53860 LEN=196
2026-05-20T22:23:26.864093+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.
121 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=55098 PROTO=UDP SPT=123 DPT=41323 LEN=196
2026-05-20T22:23:27.059509+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.
123 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=16088 PROTO=UDP SPT=123 DPT=45694 LEN=196
2026-05-20T22:23:43.965285+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s8 OUT= MAC=
DST=224.0.0.251 LEN=73 TOS=0x00 PREC=0x00 TTL=255 ID=21864 DF PROTO=UDP SPT=5353 DPT=5353 LEN=53 SRC=10.0.0.1
2026-05-20T22:23:43.965344+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=
DST=224.0.0.251 LEN=73 TOS=0x00 PREC=0x00 TTL=255 ID=55282 DF PROTO=UDP SPT=5353 DPT=5353 LEN=53 SRC=10.0.2.15
```

Monitoring logs in real time is a core SOC analyst workflow. The ability to watch a firewall's decisions live while generating traffic from another machine is an essential validation skill.

Understanding a Log Entry

Step 5.2 — Elaborate a Real Log Entry from This Lab

The following is a real log entry captured during this lab:

```
FW_INPUT_DROP: IN=enp0s3 OUT=
MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00
SRC=185.125.190.122 DST=10.0.2.15 PROTO=UDP SPT=123 DPT=46070 LEN=76
```

Field	Value	Meaning
FW_INPUT_DROP:	log prefix	A packet reached the INPUT chain and was dropped by the default DROP policy.
IN=enp0s3	WAN interface	Packet arrived on the WAN-side interface — from the external network.
OUT=	empty	No outbound interface — this was destined for Ubuntu itself, not forwarded.
SRC=185.125.190.122	external server	The source is a public internet IP — an external NTP server.
DST=10.0.2.15	Ubuntu WAN IP	The destination is Ubuntu's own WAN address.

Field	Value	Meaning
PROTO=UDP	UDP	Connectionless protocol — NTP uses UDP.
SPT=123	NTP server port	Port 123 is the standard NTP (Network Time Protocol) port.
DPT=46070	Ubuntu ephemeral port	The port Ubuntu opened when it originally sent the NTP request.

What actually happened: Ubuntu sent an NTP time-sync request to an internet NTP server. The server replied. The reply packet arrived on enp0s3 and entered the INPUT chain — but Ubuntu's ESTABLISHED/RELATED rule did not catch it, likely because the NTP session state was not tracked (UDP is connectionless). The packet hit the catch-all DROP log rule and was discarded.

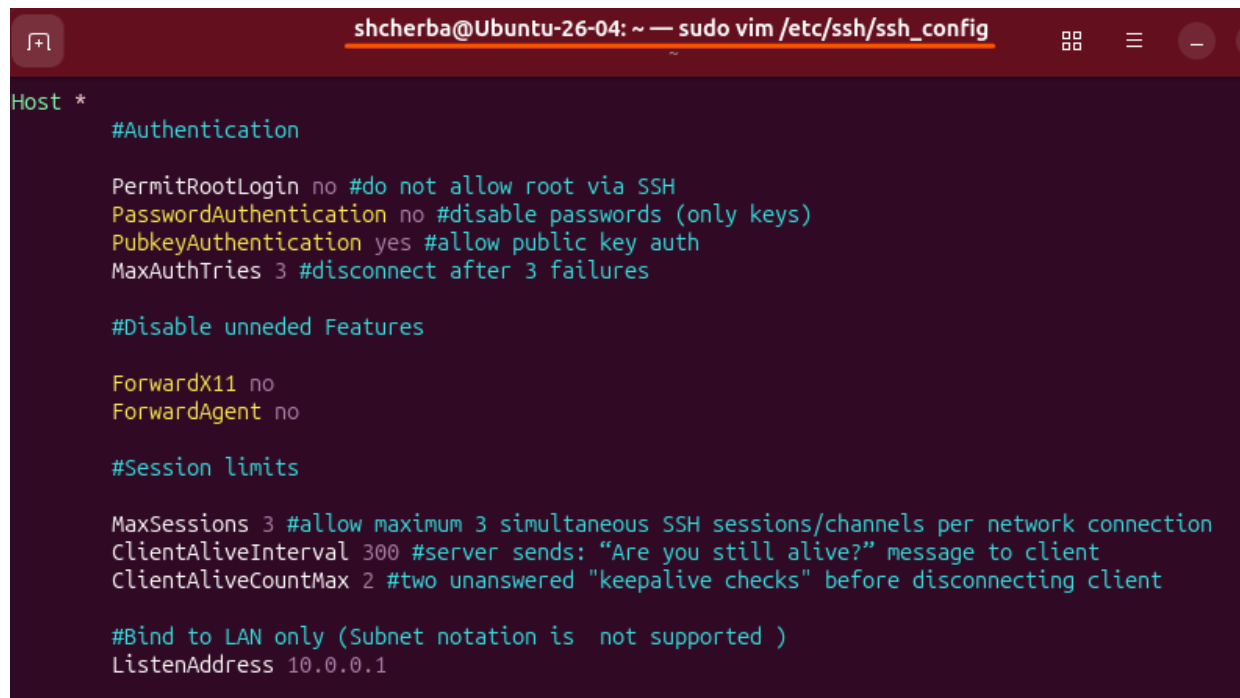
Security Significance: Dedicated firewall logs are critical for isolating security-relevant events from general system noise. The FW_ prefix enables instant grep-based filtering. In a real SOC environment, these log entries would feed into a SIEM for correlation and alerting.

06 SSH HARDENING

The firewall already restricts SSH to the LAN interface and rate-limits new connections. This section hardens the SSH daemon itself — controlling authentication methods, session behaviour, and adding an automated ban layer.

Step 6.1 — Edit the SSH Daemon Configuration File

What it does: Opens the SSH daemon configuration. Changes here control how SSH accepts connections — authentication methods, which interfaces to listen on, and session limits.



```
shcherba@Ubuntu-26-04: ~ — sudo vim /etc/ssh/sshd_config

Host *
    #Authentication

    PermitRootLogin no #do not allow root via SSH
    PasswordAuthentication no #disable passwords (only keys)
    PubkeyAuthentication yes #allow public key auth
    MaxAuthTries 3 #disconnect after 3 failures

    #Disable unneeded Features

    ForwardX11 no
    ForwardAgent no

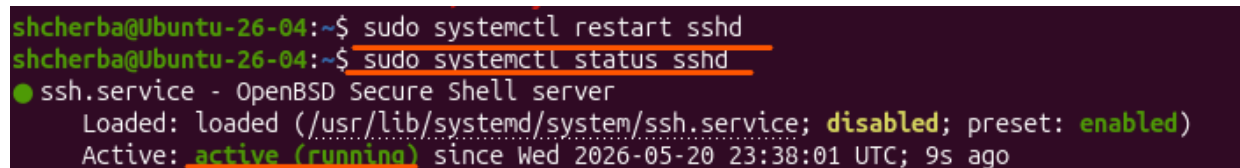
    #Session limits

    MaxSessions 3 #allow maximum 3 simultaneous SSH sessions/channels per network connection
    ClientAliveInterval 300 #server sends: "Are you still alive?" message to client
    ClientAliveCountMax 2 #two unanswered "keepalive checks" before disconnecting client

    #Bind to LAN only (Subnet notation is not supported)
    ListenAddress 10.0.0.1
```

Step 6.2 — Restart sshd to Apply Configuration Changes

What it does: Restarts the SSH daemon so it reads the updated configuration. The status check confirms it is running — if there is a syntax error in sshd_config, the daemon will refuse to start, which is caught immediately here.



```
shcherba@Ubuntu-26-04:~$ sudo systemctl restart sshd
shcherba@Ubuntu-26-04:~$ sudo systemctl status sshd
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Wed 2026-05-20 23:38:01 UTC; 9s ago
```

If sshd fails to restart, should run 'sudo sshd -t' to test the configuration file for syntax errors. It reports the exact line causing the problem.

Key-Based Authentication

Step 6.3 — Generate SSH Key Pair on Parrot

What it does: Generates an Ed25519 key pair on Parrot. '-t ed25519' selects the algorithm — Ed25519 is currently recommended by OpenSSH for its speed and security. '-C' adds a comment to

the key for identification. The private key (id_ed25519) never leaves Parrot; the public key (id_ed25519.pub) will be copied to Ubuntu.

```
[user@parrot]~  
$ ssh-keygen -t ed25519 -C "firewall_lab"  
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/user/.ssh/id_ed25519):
```

Step 6.4 — Copy the Public Key to Ubuntu

What it does: Appends Parrot's public key to Ubuntu's ~/.ssh/authorized_keys file. This is the one-time setup step — after this, SSH will use the key instead of a password. The command uses password authentication for this single transfer, which is why PasswordAuthentication must still be enabled at this stage.

```
[x]-[user@parrot]~  
$ ssh-copy-id shcherba@10.0.0.1  
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/user/.ssh/id_ed25519.pub"  
The authenticity of host '10.0.0.1 (10.0.0.1)' can't be established.  
ED25519 key fingerprint is SHA256:2aAT8BBV0D+V90Gf2GhYV5/Eb+lLm5Jkh1j+wfWBHLo.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed  
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys  
shcherba@10.0.0.1's password:  
Permission denied, please try again.  
shcherba@10.0.0.1's password:  
Number of key(s) added: 1
```

Step 6.5 — Verify the Public Key Arrived on Ubuntu

What it does: Confirms the public key is present on Ubuntu. The output should show a line beginning with 'ssh-ed25519' followed by the key content and the comment specified during generation.

```
shcherba@Ubuntu-26-04:~$ ls ~/.ssh/  
authorized_keys  
shcherba@Ubuntu-26-04:~$ cat ~/.ssh/authorized_keys  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHmoWo+kvwsUvpZJTKAzaXCL48+IuqOXXtuA89f17rtX firewall lab
```

Step 6.6 — Test Key-Based Login from Parrot

What it does: Connects to Ubuntu using key-based authentication. With PasswordAuthentication disabled in sshd_config, SSH will use the key automatically. A successful login without a password prompt confirms the key exchange is working correctly.

```
[user@parrot]-[~]
$ ssh shcherba@10.0.0.1
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

6 updates can be applied immediately.
4 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

shcherba@Ubuntu-26-04:~$
```

Fail2ban — Automated Brute-Force Protection

Step 6.7 — Install Fail2ban

What it does: Installs Fail2ban — a daemon that monitors log files for repeated authentication failures and automatically adds temporary iptables DROP rules for the offending IPs.

```
shcherba@Ubuntu-26-04:~$ sudo apt install fail2ban -y
Installing:
  fail2ban

Installing dependencies:
  python3-pyasyncore python3-pyinotify whois
```

Step 6.8 — Configure the SSH Jail

What it does: jail.local is the user override file for Fail2ban — always edit this instead of jail.conf so changes survive package updates. The [sshd] jail monitors /var/log/auth.log and bans IPs after 3 failures within 10 minutes for 1 hour.

```
shcherba@Ubuntu-26-04: ~ — sudo vim /etc/fail2ban/jail.local

[DEFAULT] #section defines global settings that apply to ALL services unless overridden.
bantime = 3600 #if someone gets banned, they are blocked for 1 hour (1 hour = 3600 seconds)
findtime = 600 #fail2ban looks at the last 10 minutes of logs
maxretry = 3 #if 3 failures happen within 10 minutes it will ban IP
#Generally, IF 3 failed attempts within 10 min --> ban for 1 hour

[sshd] #jail definition for sshd, simple label
enabled = true #monitoring for SSH
port 22 #specifies, which port SSH listens to
logpath = /var/log/iptables.log #what log file to check
maxretry = 3

#workflow virtualization:
#Attacker -> SSH attempt --> iptables.log --> Fail2ban reads logs --> detects repeated failures --> triggers iptables action --> firewall drops attacker IP
```

Step 6.9 — Start Fail2ban and Check SSH Jail Status

What it does: Enables and starts the Fail2ban service. 'fail2ban-client status sshd' queries the SSH jail's current state — showing how many IPs are currently banned and the total failure count. As specified in jail.local, the sshd jail is now actively monitoring the SSH daemon.

```
shcherba@Ubuntu-26-04:~$ sudo systemctl start fail2ban
shcherba@Ubuntu-26-04:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:    0
| `-- Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
  |- Currently banned: 0
  |- Total banned:    0
  `-- Banned IP list:
```

Defence in Depth: iptables rate-limits SSH at the kernel packet level. Fail2ban bans IPs at the application level after repeated auth failures. These two layers operate independently — an attacker who bypasses one still faces the other. Combining kernel-level and application-level controls is the essence of defence in depth.

07 VALIDATION — PORT SCANNING WITH PARROT

With the firewall fully configured, the following tests validate that each rule behaves as intended from an external perspective.

Step 7.1 — Port Scan Ubuntu with Nmap

What it does: '-sS' performs a TCP SYN scan — nmap sends SYN packets and records whether each port replies with SYN-ACK (open), RST (closed), or nothing (filtered). '-p 1-65535' scans all ports. Since Ubuntu's INPUT policy is DROP, all ports without an explicit ACCEPT rule will be reported as 'filtered'. Only port 22/tcp appeared open — this confirms the deny-by-default policy is enforced.

```
[*]-[user@parrot]-[~]
$ sudo nmap -sS -p 1-65535 10.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-21 00:36 UTC
Nmap scan report for 10.0.0.1
Host is up (0.00017s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:16:FC:A6 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 16.36 seconds
```

Step 7.2 — Check the Firewall Log on Ubuntu During the Scan

What it does: Watches the log in real time during the nmap scan. Every port probe that does not match an ACCEPT rule fires the catch-all LOG rule with the FW_INPUT_DROP: prefix. The volume of entries during a scan visually demonstrates the firewall actively processing and dropping each probe.

```
shcherba@Ubuntu-26-04:/etc/rsyslog.d$ sudo tail -f /var/log/kern.log
2026-05-21T00:44:10.607324+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.123 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=51084 PROTO=UDP SPT=123 DPT=41927 LEN=196
2026-05-21T00:44:13.311413+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=91.189.91.112 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=49004 PROTO=UDP SPT=123 DPT=46590 LEN=196
2026-05-21T00:44:15.413615+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.122 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=20953 PROTO=UDP SPT=123 DPT=46615 LEN=196
2026-05-21T00:45:20.625150+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.122 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=42039 PROTO=UDP SPT=123 DPT=51963 LEN=196
2026-05-21T00:45:21.137359+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=91.189.91.113 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=18326 PROTO=UDP SPT=123 DPT=46078 LEN=196
2026-05-21T00:45:22.049187+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.121 DST=10.0.2.15 LEN=216 TOS=0x00 PREC=0x00 TTL=64 ID=53859 PROTO=UDP SPT=123 DPT=40487 LEN=196
2026-05-21T00:46:20.933895+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= MAC=08:00:27:63:a1:97:52:54:00:12:35:00:08:00 SRC=185.125.190.123 DST=10
```

Log Entries from the Port Scan

The following entries were captured on Ubuntu during the nmap scan from Parrot:

```
2026-05-21T00:44:10.607324+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= SRC=10.0.0.2 DST=10.0.0.1 PROTO=TCP DPT=445
```

```
2026-05-21T00:44:13.311413+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= SRC=10.0.0.2 DST=10.0.0.1 PROTO=TCP DPT=80
```

```
2026-05-21T00:44:15.413615+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= SRC=10.0.0.2 DST=10.0.0.1 PROTO=TCP DPT=443
```

```
2026-05-21T00:45:20.625150+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= SRC=10.0.0.2  
DST=10.0.0.1 PROTO=TCP DPT=21  
  
2026-05-21T00:45:21.137359+00:00 Ubuntu-26-04 kernel: FW_INPUT_DROP: IN=enp0s3 OUT= SRC=10.0.0.2  
DST=10.0.0.1 PROTO=TCP DPT=25
```

What these entries confirm:

- **SRC=10.0.0.2:** Every probe originates from Parrot, confirming it is the scanning host.
- **DST=10.0.0.1:** All probes target Ubuntu's LAN IP, confirming correct network routing.
- **PROTO=TCP:** nmap's SYN scan uses TCP, as expected.
- **DPT= (various):** Ports 445 (SMB), 80 (HTTP), 443 (HTTPS), 21 (FTP), 25 (SMTP)... — all dropped. None of these services are permitted.
- **FW_INPUT_DROP prefix:** Confirms the catch-all logging rule at the bottom of the INPUT chain is firing for every unmatched probe.

Summary

This lab demonstrates a complete Linux firewall and router implementation using iptables on Ubuntu 26.04, with Parrot 7.1 KDE as the validation machine.

Objective	Result
Dual-NIC routing in VirtualBox	Configured — enp0s3 (WAN/NAT), enp0s8 (LAN/intnet)
Static IPs and default gateway	Ubuntu: 10.0.0.1/24 — Parrot: 10.0.0.2/24, gateway 10.0.0.1
IP forwarding enabled	Verified — /proc/sys/net/ipv4/ip_forward = 1, persisted via sysctl.conf
Firewall script built	27 rules across filter and nat tables — deny-by-default stance
NAT / IP Masquerade	Working — Parrot reaches internet; internet sees Ubuntu's WAN IP
Stateful filtering	ESTABLISHED/RELATED rules prevent blocking reply traffic
SSH hardened	LAN-only, rate-limited, key-based auth, Fail2ban active
Logging	FW_ prefixed log entries — dedicated rsyslog file, real-time monitoring
Segmentation	HTTP (port 80) blocked, HTTPS passes; WAN SSH completely blocked
Persistence	iptables-persistent + netfilter-persistent service enabled at boot
Validation	nmap scan from Parrot confirms deny-by-default; log entries confirm all drops are recorded

The End.